

Standalone Boot Mode — UART Bootloader

This guide shows how to program your firmware into the board over a single USB cable — no JTAG, no Vivado, no Vitis needed. The application is stored in QSPI flash and runs automatically at every power-on, exactly like a commercial MCU.

Prerequisites

- A board that has been through the **one-time setup** (§2) — lab boards are usually pre-programmed
- Python 3 with `pyserial` (`pip install pyserial`)
- An application ELF built from the `SRAM_app_template` (§3)

1. How Standalone Boot Works

Power-on → FPGA configures itself from QSPI flash (<1 s, quad-SPI x4 @ 33 MHz)

→ Bootloader (in Block RAM, part of the bitstream) starts

→ Listens on UART for ~1 s:

upload request? → receive app → write to flash → run

silence? → copy app from flash to SRAM → run

Memory roles:

Memory	Size	Role
QSPI flash	4 MB	bitstream (2.1 MB) + application slot @ <code>0x3000000</code> (non-volatile)
SRAM @ <code>0x60000000</code>	512 KB	where your application executes (code/data/heap, I/D-cached)
BRAM @ <code>0x00000000</code>	128 KB	32 KB bootloader (untouchable) · 32 KB ITCM @ <code>0x8000</code> (your fast code) · 64 KB DTCM @ <code>0x10000</code> (stack + fast data)

Note: The bootloader is part of the bitstream, so it is restored from flash at every power-on — like a mask ROM, it cannot be bricked from software. Holding the on-board button while plugging in USB forces the board to stay in the bootloader.

2. One-Time Board Setup (Instructor)

Program the deployment image `release/boot.mcs` (bitstream + bootloader) into the QSPI flash:

1. Open Vivado **Hardware Manager > Open Target > Auto Connect**.
2. Right-click `xc7a35t_0` > **Add Configuration Memory Device** — pick the part matching the IC3 marking: `mx25l3273f-spi-x1_x2_x4` (Macronix, verified) or `n25q32-3.3v-spi-x1_x2_x4` (Micron, older boards).

3. Right-click the memory device > **Program Configuration Memory Device**, select `release/boot.mcs`, keep **Erase / Program / Verify** checked, **OK**.
4. Power-cycle. LED0 lights (bootloader alive), then blinks slowly (no app yet).

Students never repeat this step; from here on the board is a pure MCU.

3. Build Your Application

Start from `workspace-example/SRAM_app_template/` (copy `main.c` and `lscript.ld` into a Vitis application as in the JTAG guide). The template's linker script places:

- `.text` / `.rodata` / `.data` / `.bss` / heap → **SRAM** (512 KB, cached)
- stack → top of **DTCM** (64 KB BRAM @ `0x10000`; 1-cycle, never collides with the bootloader)
- functions tagged `ITCM_FUNC` → **ITCM** (32 KB BRAM @ `0x8000`); variables tagged `DTCM_DATA` → DTCM

ITCM code ships inside the SRAM image and is copied out by `tcm_init()` at the top of `main()` (keep that call — it is the same startup-copy idiom an STM32H7 uses for its ITCM). Put interrupt handlers and timing-critical loops there: TCM never misses, so worst-case latency equals best-case.

Check the footprint after building: the image must fit in 512 KB, or the linker reports a region overflow.

Note: The same ELF works unchanged in **both** modes — Vitis **Run/Debug** over JTAG during development, `upload.py` for deployment. No rebuild, no reconfiguration.

4. Upload

```
python3 tools/upload.py build/<app>.elf          # write to flash, then run
python3 tools/upload.py build/<app>.elf --ram     # test run from RAM (flash
untouched)
```

Power-cycle the board (or plug it in) when the script says it is syncing — the bootloader listens during its 1-second boot window. Typical upload takes a few seconds (chunked transfer with CRC32 verification, then sector-erase + page-program + read-back verify).

Useful flags: `--port /dev/ttyUSBx` if auto-detection picks the wrong port, `--entry` for raw `.bin` files.

5. Boot and Verify

- **Power-cycle** → the board boots your flashed app automatically. No PC needed.
- `xil_printf` output arrives on the same USB cable at **115200 baud** (the template sets this; the bootloader uses the same rate, so one terminal session covers both).
- Convention: light an LED first thing in `main()` so "configured, booted, and running" is visible at a glance.

Note: The reset button restarts the CPU into the **bootloader** (BRAM is intact), which reloads your app from flash — so reset $\approx$ power-cycle in this design. Modified `.data` globals are restored because the image is re-copied from flash each boot.

6. Working with JTAG Debug Mode

Both modes coexist on the same board with no switching:

	JTAG (Vitis Run/Debug)	Standalone (<code>upload.py</code>)
Code goes to	RAM directly (volatile)	Flash (persistent)
After power-cycle	back to the flashed app	your app boots
Breakpoints / stepping	yes	no
Needs	Vivado/Vitis installed	Python + USB cable

Daily development loop: **edit** $\rightarrow$ **build** $\rightarrow$ **JTAG Run/Debug**; when it works, `upload.py` once to "ship" it. JTAG always has priority — it halts the CPU before the bootloader runs, so nothing in flash can interfere with a debug session.

7. How the Deployment Image Is Made (Instructor Reference)

`release/boot.mcs` = bitstream with the bootloader merged into its BRAM initialization:

```
# 1. build workspace-example/bootloader/ in Vitis (linker confines it to the lower
32 KB BRAM)
# 2. merge the ELF into the bitstream BRAM init:
tools/make_boot_mcs.sh bootloader.elf --hw release/top_wrapper/
# (wraps: updatemem -proc top_i/microblaze_riscv_0 + write_cfgmem -interface
SPIx4)
```

The bootloader source (`workspace-example/bootloader/src/bootloader.c`, ~250 lines) is deliberately readable — it doubles as course material for: reset vectors, loaders, UART protocols, SPI flash command sequences (WREN/erase/program/status-poll), CRC verification, and `fence.i`.

8. Troubleshooting

1. **`upload.py` can't sync** — Power-cycle during the sync window; or hold the button while plugging in (forces bootloader mode). Check the port: the FT232 exposes two — pick the UART one (`--port`). Close any open serial terminal first (port is exclusive).
2. **Upload OK but app misbehaves** — Ensure the app was built with the template's `lscript.ld` (sections must live in SRAM; a default all-BRAM ELF uploads but its addresses collide with the bootloader). `upload.py` warns about segments outside SRAM.

3. **Board boots an old app** — The upload failed before the `G` step, or you used `--ram` (volatile). Re-run without `--ram` and watch for the final "programmed to flash" message.
4. **Nothing at all (no LED)** — FPGA didn't configure: flash image invalid → redo §2. JTAG always works for recovery.
5. **Garbage on the terminal** — Baud must be **115200** (both bootloader and template), not 9600.
6. **Want a factory-blank board** — Hardware Manager > right-click memory device > **Erase**: the FPGA then waits for JTAG at power-on.